

Complex Hackathon Example: The "Cascading API Mesh Failure" Resolution

To demonstrate the full breadth of the Autonomous Incident Resolution Platform (AIRP), we will implement a scenario involving a **cascading failure across a microservices API mesh** where the root cause is initially obscured by symptoms in downstream services.

🌟 The Scenario: "Checkout Latency Spiral"

Context: A core e-commerce platform uses 5 microservices deployed on Kubernetes. The **Checkout Service** (S1) suddenly experiences a massive spike in latency. Initial alerts point directly to S1.

The Reality (Hidden Complexity):

1. **S1 (Checkout Service):** High Latency (Symptom 1).
2. **S1 depends on S2 (Inventory Service):** S2 is not down, but its database connection pool is exhausted.
3. **S2 depends on S3 (Pricing Service):** S3 is running normally but is experiencing a minor memory leak, causing GC pauses.
4. **S4 (Payment Gateway Proxy) depends on S2:** When S2 struggles, S4 starts queuing requests, leading to resource pressure.
5. **S5 (Recommendation Service) is also dependent on S2:** S5 starts timing out on its calls to S2, creating an additional, secondary load spike on S2.

The Challenge for the AIRP: A simple monitoring alert (high latency on S1) is insufficient. The AIRP must correlate S1's latency with S2's resource metrics, S3's GC activity, and S5's timeout counts to correctly pinpoint S3 as the *actual* root cause.

🔧 Implementation Breakdown (Hackathon Steps)

The goal is to automate the resolution cycle from the initial alert to the final RCA report within 5 minutes.

Phase 1: Observation and Detection (Monitoring Agent)

- **Input:** Prometheus/Grafana alerts fired by OpenTelemetry metrics.
- **Trigger:** `Checkout_Latency > 500ms` (Alert fired on S1).
- **Action:** Monitoring Agent captures the alert payload and immediately triggers the workflow (via Temporal/Kafka).

- **Complexity Layer:** The agent doesn't just report S1 latency; it fetches the associated service tags, dependent services (S2, S3, S5), and the time window of the spike.

Phase 2: Correlation and Dependency Mapping (Correlation Agent)

- **Input:** S1 Alert + Service Dependency Graph (from the system metadata).
- **Action:** Correlation Agent queries Prometheus/Grafana for the state of all linked services (S1 $\rightarrow$ S2 $\rightarrow$ S3, S2 $\rightarrow$ S4, S2 $\rightarrow$ S5).
- **Output:** A correlated incident object:
 - *Impacted Services:* S1, S2, S4, S5 (Direct & Indirect)
 - *Primary Symptom:* High Latency on S1.
 - *Underlying Strain:* S2 Connection Pool Saturation.
 - *Potential Root Cause Candidate:* S3 GC Pause/Resource Starvation.

Phase 3: AI-Driven Root Cause Analysis (RCA Agent)

- **Input:** The correlated incident object.
- **Action:** RCA Agent uses a sophisticated GPT prompt (e.g., RAG-enabled) fed with historical operational data (runbooks, previous incident reports, service documentation).
 - **Prompt Strategy:** "Given that S1 latency spiked, and S2's connection pool is saturated, and S3 exhibits high GC activity, determine the most probable root cause, considering the dependency chain S1 $\rightarrow$ S2 $\leftarrow$ S3."
- **Output:** A ranked list of probable causes with confidence scores:
 - **P1:** S3 Memory Leak/GC Pauses (Confidence: 88%)
 - **P2:** S2 Database Saturation (Confidence: 75%)
 - **P3:** S5 Unstable Timeout Load (Confidence: 55%)

Phase 4: Remediation Recommendation and Governance (Remediation Agent + Governance Layer)

- **Input:** P1(S3 GC Pauses) and the incident context.
- **Action:** Remediation Agent queries a knowledge base of known runbooks for "S3 High GC."
 - **Recommendation:** "Initiate a rolling restart of S3 instances OR Scale up S3 replicas by 2."
- **Governance Checkpoint:**
 - **Policy:** Is the suggested action (*Scale up S3*) within the current budget/capacity policy? (Yes)
 - **Safety:** Does the action have a known rollback path? (Yes - *Scale down S3* to previous state)
 - **Confidence Score:** 88% (Acceptable threshold: 75%).
 - **Outcome: Human Approval Required.** The Temporal workflow pauses, sends a Slack/Jira notification to the SRE team, presenting the recommendation and risk profile.

Phase 5: Execution, Validation, and Documentation

- **Action:** SRE team approves the recommendation. The Workflow Orchestrator (Temporal) triggers the execution.
- **Execution Layer:** A K8s operator executes the `kubectl scale deployment/s3 --replicas=N+2` command.
- **Validation:** Monitoring Agent immediately tracks S1 latency and S2 connection pool metrics.
 - *Validation Check:* Is `S1_Latency` below 300ms for 60 seconds? (Yes). Is `S2_Pool_Utilization` below 70%? (Yes).
 - **Outcome:** Recovery is confirmed!
- **Documentation Agent:** Automatically generates the final report:
 - **Incident Summary:** Checkout Latency Spiral.
 - **Detected Symptom:** High latency on S1 (500ms).
 - **Root Cause:** Memory Leak/GC Pauses on S3.
 - **Action Taken:** Scaled S3 replicas by 2.
 - **Recovery Time:** 3 minutes 45 seconds.
 - **Confidence Score:** 88%.



Why this is complex and perfect for a Hackathon

1. **Full AIOps Loop:** It doesn't just solve a symptom; it finds the cause, recommends a fix, executes it, and validates the success—all autonomously.
2. **Multi-Agent Interaction:** It requires the coordinated effort of *at least* five distinct logical components (Monitor $\rightarrow$ Correlate $\rightarrow$ RCA $\rightarrow$ Remediate $\rightarrow$ Document).
3. **Integration Points:** It forces the use of the entire stack (Prometheus/Grafana $\rightarrow$ Kafka/NATS $\rightarrow$ Temporal $\rightarrow$ LangGraph/CrewAI $\rightarrow$ GPT $\rightarrow$ K8s Operator).
4. **The "Aha!" Moment:** The complexity isn't in the execution; it's in the **reasoning**. The jump from "S1 is slow" to "S3 is leaking memory" is a non-trivial AI task that showcases the core value proposition of the platform.
5. **Governance Demo:** The mandatory human approval pause demonstrates that the system is *smart* but *safe*, which is crucial for enterprise adoption.